RX

GUÍA INTERACTIVA

Redux, Redux Toolkit, Saga y Zustand

Estado global de punta a punta: Redux clásico, Redux Toolkit con RTK Query, Redux Saga y Zustand, comparados.

Versión PDF · 30 de octubre de 2026
12 capítulos · Los ejemplos interactivos están en la app (npm run dev)

Índice

Empezá acá

1. Cómo usar esta guía

Parte 1 · Redux clásico

2. Store, acciones y reducers
3. React Redux: hooks y connect
4. Middleware y thunks

Parte 2 · Redux Toolkit

5. configureStore y createSlice
6. createAsyncThunk y compañía
7. RTK Query

Parte 3 · Redux Saga

8. Generadores y efectos
9. Patrones: race, fork y cancel

Parte 4 · Zustand

10. Zustand

Parte 5 · Comparar y decidir

11. Las cuatro, lado a lado
12. Chuleta y preguntas de entrevista

Empezá acá

Qué estado necesita una librería (y cuál no) y cómo está armada la guía.

Capítulo 1

Cómo usar esta guía

Estado local, compartido, del servidor y global: qué herramienta va con cada uno.

Esta guía recorre las cuatro formas de manejar **estado global** que más vas a encontrar en ofertas de trabajo y en proyectos reales: **Redux** "clásico" (el que seguro viste en código legacy), **Redux Toolkit** (la forma oficial de escribir Redux hoy), **Redux Saga** (para efectos async complejos) y **Zustand** (la alternativa liviana que más creció).

Todas las demos usan el mismo tema que la guía de React: **postulaciones a trabajos**. Así, cuando compares las cuatro herramientas, lo único que cambia es la herramienta.

Antes de agarrar una librería

No todo el estado necesita una librería. La pregunta clave es **de dónde viene** el dato:

Tipo de estado	Ejemplos	Herramienta
Local de un componente	Un input, si un modal está abierto	`useState`, `useReducer`
Compartido por un subárbol	Tema, usuario logueado	Context (o subir el estado)

Tipo de estado	Ejemplos	Herramienta
Del servidor (caché de datos remotos)	Lista de ofertas, perfil del usuario	TanStack Query, RTK Query, SWR, Server Components
Global del cliente	Carrito, filtros de varias pantallas, borradores, preferencias	Redux Toolkit, Zustand

> ⚠️ **El error más común**
>
> Meter **todo** en Redux: hasta el texto de cada input y los datos que vienen de la API (con su `loading`, `error`, caché y refetch hechos a mano). Hoy los datos del servidor van a una librería de **caché** (RTK Query o TanStack Query) y en el store global queda poco: lo que realmente es del cliente y compartido.

Cómo está armada

1. **Redux clásico**: los conceptos (store, acciones, reducers), react-redux con hooks y con `connect`, y middleware con thunks. Es lo que vas a encontrar en proyectos de 2016 a 2020.
2. **Redux Toolkit**: `configureStore`, `createSlice`, `createAsyncThunk` y RTK Query. La forma recomendada de escribir Redux.
3. **Redux Saga**: generadores y efectos, y los patrones donde brilla (cancelación, carreras, polling).
4. **Zustand**: un store en un hook, sin Provider ni boilerplate.
5. **Comparación**: la misma feature hecha cuatro veces, con una tabla para decidir.

Cada demo crea **su propio store** (salvo la de Zustand, que es global como en una app real), así que el botón **Reiniciar** vuelve todo a cero.

> 💡 **Redux DevTools**
>
> Instalá la extensión **Redux DevTools** en tu navegador: con Redux Toolkit se conecta sola y te muestra cada acción, el estado antes y después, y te deja "viajar en el tiempo". Zustand también se puede conectar con su middleware `devtools`.

> ℹ️ **Versiones**
>
> Escrita en octubre de 2026 con Redux 5, React Redux 9, Redux Toolkit 2, Redux Saga 1.5 y Zustand 5.

Parte 1 · Redux clásico

Los conceptos de Redux y cómo se escribía antes de Redux Toolkit: lo que vas a ver en código legacy.

Capítulo 2

Store, acciones y reducers

Las tres ideas de Redux con un store sin React: dispatch, reducer y subscribe.

Redux se apoya en tres ideas. Si las entendés, el resto (Toolkit, Saga, incluso Zustand) son variaciones:

1. **Un solo store** guarda todo el estado global de la app, en un objeto.
2. El estado **no se modifica directo**: se **despacha una acción**, un objeto que describe qué pasó (`{ type: 'postulaciones/agregada', payload: ... }`).
3. Un **reducer**, una función pura `(estado, acción) => estadoNuevo`, calcula el estado siguiente.



Es el mismo patrón que `useReducer`, pero para toda la app y por fuera de React.

Un store sin React

DEMO **Redux puro: dispatch, reducer y subscribe**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/estado/ReduxVanilla.tsx`

```
import { useEffect, useState } from 'react'
import { crearStore, type Action, type State } from './redux-
  vanilla/store'

// Redux sin React: el store, dispatch y subscribe. Así funciona por
  dentro react-redux.
export default function ReduxVanilla() {
  const [store] = useState(crearStore) // un store nuevo por demo (se
    reinicia con el botón)
  const [state, setState] = useState<State>(store.getState())
  const [log, setLog] = useState<string[]>([])
  const [empresa, setEmpresa] = useState('')

  useEffect(() => {
    // subscribe avisa después de CADA dispatch. Devuelve la función
      para desuscribirse.
    return store.subscribe(() => setState(store.getState()))
  }, [store])

  function despachar(action: Action) {
    store.dispatch(action)
    setLog((l) => [JSON.stringify(action), ...l].slice(0, 6))
  }

  return (
    <div className="stack">
      <form
        className="row"
        onSubmit={(e) => {
          e.preventDefault()
          if (!empresa.trim()) return
          despachar({ type: 'postulaciones/agregada', payload: { id:
            Date.now(), empresa: empresa.trim() } })
          setEmpresa('')
        }}
      >
        <input value={empresa} onChange={(e) =>
          setEmpresa(e.target.value)} placeholder="Empresa" />
```

```
        <button type="submit">dispatch(agregada)</button>
        <input
          value={state.filtro}
          onChange={(e) => despachar({ type: 'filtro/cambiado',
            payload: e.target.value })}
          placeholder="Filtro"
        />
      </form>
      <ul>
        {state.postulaciones
          .filter((p) =>
            p.empresa.toLowerCase().includes(state.filtro.toLowerCase(
            )))
          .map((p) => (
            <li key={p.id}>
              {p.empresa} · {p.etapa}{' '}
              <button onClick={() => despachar({ type:
                'postulaciones/avanzada', payload: p.id
                })}>avanzar</button>
            </li>
          ))}
      </ul>
      <div className="grid-2">
        <div>
          <strong>Acciones despachadas</strong>
          <ol className="log">{log.length ? log.map((l, i) => <li key=
            {i}>{l}</li>) : <li>(ninguna)</li>}</ol>
        </div>
        <div>
          <strong>store.getState()</strong>
          <pre className="log">{JSON.stringify(state, null, 2)}</pre>
        </div>
      </div>
    </div>
  )
}
```

src/demos/estado/redux-vanilla/store.ts

```
import { legacy_createStore as createStore } from 'redux'
import { siguienteEtapa, type Postulacion } from '../api'

// 1) El estado: un objeto con TODO lo que la app necesita recordar.
export type State = { postulaciones: Postulacion[]; filtro: string }

const estadoInicial: State = {
  postulaciones: [{ id: 1, empresa: 'Nubelia', etapa: 'entrevista' }],
  filtro: '',
}
```

```
// 2) Las acciones: objetos que describen "qué pasó". El type es
  obligatorio.
export type Action =
  | { type: 'postulaciones/agregada'; payload: { id: number; empresa:
    string } }
  | { type: 'postulaciones/avanzada'; payload: number }
  | { type: 'filtro/cambiado'; payload: string }

// 3) El reducer: una función PURA (estado, acción) => estado nuevo.
  Nunca muta.
export function reducer(state: State = estadoInicial, action: Action):
  State {
  switch (action.type) {
    case 'postulaciones/agregada':
      return {
        ...state,
        postulaciones: [...state.postulaciones, { ...action.payload,
          etapa: 'postulado' }],
      }
    case 'postulaciones/avanzada':
      return {
        ...state,
        postulaciones: state.postulaciones.map((p) =>
          p.id === action.payload ? { ...p, etapa:
            siguienteEtapa[p.etapa] } : p,
        ),
      }
    case 'filtro/cambiado':
      return { ...state, filtro: action.payload }
    default:
      return state // acciones que no son de este reducer: el estado
        no cambia
  }
}

// 4) El store: guarda el estado, recibe acciones con dispatch y avisa
  a los suscriptores.
export function crearStore() {
  return createStore(reducer)
}
```

El store tiene solo tres métodos que importan:

Método	Qué hace
`store.getState()`	Devuelve el estado actual.
`store.dispatch(acción)`	Le pasa la acción al reducer y guarda el resultado como estado nuevo.

Método	Qué hace
`store.subscribe(fn)`	Llama a `fn` después de cada dispatch. Devuelve la función para desuscribirse.

Mirá la pestaña `store.ts`: el estado, las acciones (una unión discriminada por `type`), el reducer y el store.

Las reglas del reducer

> ⚠️ **El reducer tiene que ser puro**
>
> - **No muta**: devuelve un objeto nuevo (`{ ...state, ... }`, `map`, `filter`). Redux detecta cambios comparando referencias: si mutás, la UI no se entera.
> - **No tiene efectos**: nada de `fetch`, `Date.now()`, `Math.random()` ni `localStorage`. Lo impuro va antes del dispatch (en el handler o en un middleware).
> - **Devuelve el estado sin cambios** para las acciones que no le corresponden (el `default` del switch).

Esa pureza es lo que habilita lo bueno de Redux: el time travel de las DevTools, reproducir un bug con la lista de acciones, testear el reducer sin renderizar nada.

Nombres de las acciones

La convención actual es `dominio/eventoEnPasado`: `postulaciones/agregada`, `filtro/cambiado`. Describen **qué pasó**, no qué hacer (`SET_FILTRO`). En código viejo vas a ver `ADD_TODO` en mayúsculas, con constantes aparte; es lo mismo con otro estilo.

ℹ️ legacy_createStore

En Redux 5, `createStore` aparece tachado (deprecado) para empujarte a Redux Toolkit. Sigue funcionando. `legacy_createStore` es el mismo, sin el tachado: lo usamos acá para mostrar Redux "a mano".

🎤 En una entrevista

"¿Cuáles son los principios de Redux?" → "Una sola fuente de verdad (el store), el estado es de solo lectura y solo cambia despachando acciones, y los cambios los calculan reducers puros. Eso hace el flujo predecible: puedo ver cada acción, reproducirla y testear la lógica sin la UI."

Capítulo 3

React Redux: hooks y connect

Provider, useSelector, useDispatch, selectores memorizados y connect().

`react-redux` conecta el store con los componentes. Hace tres cosas: pone el store disponible en todo el árbol (`Provider`), te deja **leer** una parte del estado (`useSelector`) y **despachar** acciones (`useDispatch`). Por debajo usa `useSyncExternalStore`, el mismo hook que vimos en la guía de React.

DEMO **Provider, useSelector, useDispatch y connect**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/estado/ReactRedux.tsx`

```
import { useState } from 'react'
import { Provider } from 'react-redux'
import { useRenderFlash } from '../shared/useRenderFlash'
import { useAppDispatch, useAppSelector } from './react-redux/hooks'
import { selectFiltradas, selectFiltro, selectTotal } from './react-
  redux/selectors'
import TotalConectado from './react-redux/TotalConectado'
import { crearStore } from './redux-vanilla/store'

export default function ReactRedux() {
  const [store] = useState(crearStore)
  // El Provider pone el store a disposición de todo el árbol (por
    context).
  return (
    <Provider store={store}>
      <div className="stack">
        <Filtro />
        <div className="grid-2">
          <Lista />
```

```
          <Total />
        </div>
        <TotalConectado />
      </div>
    </Provider>
  )
}

const nuevas = ['Pagos Sur', 'Andes Labs', 'Mate Cloud', 'Tereré
  Tech']

function Filtro() {
  const filtro = useAppSelector(selectFiltro)
  const dispatch = useAppDispatch()
  const ref = useRenderFlash<HTMLDivElement>()
  return (
    <div ref={ref} className="row card">
      <input
        value={filtro}
        onChange={(e) => dispatch({ type: 'filtro/cambiado', payload:
          e.target.value })}
        placeholder="Filtrar"
      />
      <button
        onClick={() => {
          const empresa = nuevas[Math.floor(Math.random() *
            nuevas.length)]
          dispatch({ type: 'postulaciones/agregada', payload: { id:
            Date.now(), empresa } })
        }}
      >
        Agregar una
      </button>
    </div>
  )
}

function Lista() {
  // useSelector re-renderiza el componente solo si lo que devuelve el
    selector cambió (===).
  const postulaciones = useAppSelector(selectFiltradas)
  const dispatch = useAppDispatch()
  const ref = useRenderFlash<HTMLDivElement>()
  return (
    <div ref={ref} className="card">
      <strong>Lista</strong>
      <ul>
        {postulaciones.map((p) => (
          <li key={p.id}>
            {p.empresa} · {p.etapa}{' '}
```

```
            <button onClick={() => dispatch({ type:
              'postulaciones/avanzada', payload: p.id
              })}>avanzar</button>
          </li>
        ))}
      </ul>
    </div>
  )
}

function Total() {
  // Este componente solo lee el total: escribir en el filtro NO lo
    re-renderiza.
  const total = useAppSelector(selectTotal)
  const ref = useRenderFlash<HTMLDivElement>()
  return (
    <div ref={ref} className="card">
      <strong>Total</strong>
      <p style={{ fontSize: '2rem', margin: 0 }}>{total}</p>
    </div>
  )
}
```

src/demos/estado/react-redux/selectors.ts

```
import { createSelector } from 'reselect'
import type { State } from '../redux-vanilla/store'

// Selectores: funciones que leen (o calculan) algo del estado.
export const selectFiltro = (state: State) => state.filtro
export const selectPostulaciones = (state: State) =>
  state.postulaciones
export const selectTotal = (state: State) =>
  state.postulaciones.length

// createSelector memoriza: solo recalcula si cambian sus entradas.
  Así devuelve
// el MISMO array mientras nada cambie, y useSelector no re-renderiza
  de más.
export const selectFiltradas = createSelector([selectPostulaciones,
  selectFiltro], (postulaciones, filtro) =>
  postulaciones.filter((p) =>
    p.empresa.toLowerCase().includes(filtro.toLowerCase())),
)
```

src/demos/estado/react-redux/hooks.ts

```
import { useDispatch, useSelector } from 'react-redux'
import type { Dispatch } from 'redux'
```

```ts
import type { Action, State } from '../redux-vanilla/store'

// Versiones tipadas de los hooks: así no hay que escribir los tipos
  en cada componente.
export const useAppSelector = useSelector.withTypes<State>()
export const useAppDispatch = useDispatch.withTypes<Dispatch<Action>>
  ()
```

src/demos/estado/react-redux/TotalConectado.tsx

```tsx
import { connect, type ConnectedProps } from 'react-redux'
import type { Action, State } from '../redux-vanilla/store'

// La forma "legacy": connect() con mapStateToProps y
  mapDispatchToProps.
// La vas a encontrar en proyectos viejos. En código nuevo, usá los
  hooks.
const mapStateToProps = (state: State) => ({ total:
  state.postulaciones.length })
const mapDispatchToProps = {
  limpiarFiltro: (): Action => ({ type: 'filtro/cambiado', payload: ''
    }),
}

const conector = connect(mapStateToProps, mapDispatchToProps)

function Total({ total, limpiarFiltro }: ConnectedProps<typeof
  conector>) {
  return (
    <p>
      Total (con connect): <strong>{total}</strong> <button onClick=
        {limpiarFiltro}>Limpiar filtro</button>
    </p>
  )
}

const TotalConectado = conector(Total)
export default TotalConectado
```

Escribí en el filtro y mirá los recuadros que parpadean: **solo se re-renderizan los componentes cuyo selector devolvió algo distinto**. `Total` lee solo la cantidad, así que filtrar no lo toca. Agregá una empresa y ahí sí parpadea.

useSelector

```
const filtro = useAppSelector((state) => state.filtro)
```

- Corre el selector en cada dispatch y compara el resultado con el anterior usando `===`.
- Si es igual, **no re-renderiza**. Por eso conviene seleccionar lo mínimo: `state.postulaciones.length` en vez de `state.postulaciones` si solo mostrás el total.
- Podés llamar a `useSelector` varias veces en un componente, una por dato.

> ⚠️ **Selectores que devuelven algo nuevo cada vez**
>
> `useSelector((s) => s.postulaciones.filter(...))` devuelve **un array nuevo en cada llamada**, aunque nada haya cambiado. Resultado: el componente se re-renderiza con **cualquier** acción. React Redux incluso te avisa en la consola en desarrollo. La solución es **memorizar** el selector con `createSelector` (de Reselect, incluido en Redux Toolkit), como `selectFiltradas` en la demo.

Selectores memorizados

`createSelector([entrada1, entrada2], calculo)` solo vuelve a ejecutar `calculo` si alguna entrada cambió. Si no, devuelve **el mismo resultado de antes** (la misma referencia), y `useSelector` no re-renderiza. Es el equivalente a `useMemo`, pero para datos derivados del store.

Definir los selectores en un archivo aparte (`selectors.ts`) también ayuda a que los componentes no conozcan la forma exacta del estado.

Hooks tipados

Con TypeScript, en vez de tipar `state` en cada `useSelector`, se crean una vez versiones tipadas:

```
export const useAppSelector = useSelector.withTypes<RootState>()
export const useAppDispatch = useDispatch.withTypes<AppDispatch>()
```

connect: la forma legacy

Antes de los hooks (2019), la única forma era `connect`, un *higher-order component*. Lo vas a encontrar en muchísimo código:

src/demos/estado/react-redux/TotalConectado.tsx

```
import { connect, type ConnectedProps } from 'react-redux'
import type { Action, State } from '../redux-vanilla/store'

// La forma "legacy": connect() con mapStateToProps y
  mapDispatchToProps.
// La vas a encontrar en proyectos viejos. En código nuevo, usá los
  hooks.
const mapStateToProps = (state: State) => ({ total:
  state.postulaciones.length })
const mapDispatchToProps = {
  limpiarFiltro: (): Action => ({ type: 'filtro/cambiado', payload: ''
    }),
}

const conector = connect(mapStateToProps, mapDispatchToProps)

function Total({ total, limpiarFiltro }: ConnectedProps<typeof
  conector>) {
  return (
    <p>
      Total (con connect): <strong>{total}</strong> <button onClick=
        {limpiarFiltro}>Limpiar filtro</button>
    </p>
  )
}

const TotalConectado = conector(Total)
export default TotalConectado
```

- `mapStateToProps(state)` elige qué parte del estado llega como props.
- `mapDispatchToProps` (un objeto de action creators o una función) convierte acciones en props que despachan.
- `connect(map1, map2)(Componente)` devuelve un componente nuevo que recibe esas props.

`connect`	Hooks
Envuelve el componente (HOC)	Se llaman adentro del componente
El componente recibe todo por props	Cada dato se lee donde se usa
Tipar `ConnectedProps` es más trabajoso	Inferencia simple con `withTypes`
Sigue soportado	**Recomendado** para código nuevo

> 💡 **Migrar de connect a hooks**
>
> No hace falta migrar todo de una. Los dos conviven en el mismo store. Una estrategia común: cada vez que tocás un componente con `connect`, lo pasás a hooks.

> 🎤 **En una entrevista**
>
> "¿Cuándo se re-renderiza un componente con useSelector?" → "Cuando el valor que devuelve el selector cambia por referencia (===) después de un dispatch. Por eso selecciono lo mínimo y memorizo con createSelector los selectores que devuelven arrays u objetos derivados; si no, el componente se re-renderiza con cualquier acción."

Capítulo 4

Middleware y thunks

Dónde va lo async en Redux, un logger a mano y el problema del boilerplate.

El reducer tiene que ser puro, así que no puede hacer un `fetch`. ¿Dónde va entonces lo async? En un **middleware**: una función que se mete entre el `dispatch` y el reducer y puede hacer lo que quiera con cada acción: anotarla, transformarla, demorarla o despachar otras.

```
dispatch(acción) ⟶ middleware 1 ⟶ middleware 2 ⟶ reducer
```

Un middleware por dentro

Un middleware tiene una firma rara, con tres funciones anidadas:

```
const logger: Middleware = (store) => (next) => (action) => {
  console.log('antes', store.getState())
  const resultado = next(action) // sigue la cadena (al final, el
    reducer)
  console.log('después', store.getState())
  return resultado
}
```

- `store` tiene `getState` y `dispatch`.
- `next` pasa la acción al siguiente middleware.
- Si un middleware no llama a `next`, la acción no llega al reducer.

Thunks: lo async más simple

`redux-thunk` es un middleware de 10 líneas: si lo que despachás es una **función** en vez de un objeto, la ejecuta pasándole `dispatch` y `getState`. Adentro podés hacer `await` y despachar las acciones que quieras.

DEMO **Un thunk con estados de carga y un middleware logger**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/estado/ThunkClasico.tsx`

```
import { useState } from 'react'
import { Provider, useDispatch, useSelector } from 'react-redux'
import { cargarPostulaciones, crearStore, type AppDispatch, type State
  } from './thunk/store'

export default function ThunkClasico() {
  const [log, setLog] = useState<string[]>([])
  const [store] = useState(() => crearStore((linea) => setLog((l) =>
    [...l, linea])))
  return (
    <Provider store={store}>
      <div className="stack">
        <Postulaciones />
        <ol className="log">
          {log.length ? log.map((l, i) => <li key={i}>{l}</li>) : <li>
            (el logger todavía no vio nada)</li>}
        </ol>
      </div>
    </Provider>
  )
}

function Postulaciones() {
  const { postulaciones, status, error } = useSelector((s: State) =>
    s)
  const dispatch = useDispatch<AppDispatch>()
  return (
    <>
      <div className="row">
        <button onClick={() =>
          dispatch(cargarPostulaciones(false))}>Cargar</button>
```

```
        <button onClick={() =>
          dispatch(cargarPostulaciones(true))}>Cargar (con error)
          </button>
        <span className="pill">status: {status}</span>
      </div>
      {status === 'loading' && <p>Cargando…</p>}
      {error && <p className="error-text">{error}</p>}
      <ul>
        {postulaciones.map((p) => (
          <li key={p.id}>
            {p.empresa} · {p.etapa}
          </li>
        ))}
      </ul>
    </>
  )
}
```

src/demos/estado/thunk/store.ts

```
import { applyMiddleware, legacy_createStore as createStore, type
  Middleware } from 'redux'
import { thunk, type ThunkAction, type ThunkDispatch } from 'redux-
  thunk'
import { api, type Postulacion } from '../api'

// Redux "a mano", como en muchos proyectos de 2016-2019: constantes,
  action creators y thunks.
export const CARGA_INICIADA = 'postulaciones/cargaIniciada'
export const CARGA_EXITOSA = 'postulaciones/cargaExitosa'
export const CARGA_FALLIDA = 'postulaciones/cargaFallida'

export type State = {
  postulaciones: Postulacion[]
  status: 'idle' | 'loading' | 'succeeded' | 'failed'
  error: string | null
}

type Action =
  | { type: typeof CARGA_INICIADA }
  | { type: typeof CARGA_EXITOSA; payload: Postulacion[] }
  | { type: typeof CARGA_FALLIDA; error: string }

// Action creators: funciones que arman la acción (para no escribir el
  objeto a mano cada vez).
const cargaIniciada = (): Action => ({ type: CARGA_INICIADA })
const cargaExitosa = (postulaciones: Postulacion[]): Action => ({
  type: CARGA_EXITOSA, payload: postulaciones })
const cargaFallida = (error: string): Action => ({ type:
  CARGA_FALLIDA, error })
```

```
const inicial: State = { postulaciones: [], status: 'idle', error:
  null }

function reducer(state: State = inicial, action: Action): State {
  switch (action.type) {
    case CARGA_INICIADA:
      return { ...state, status: 'loading', error: null }
    case CARGA_EXITOSA:
      return { ...state, status: 'succeeded', postulaciones:
        action.payload }
    case CARGA_FALLIDA:
      return { ...state, status: 'failed', error: action.error }
    default:
      return state
  }
}

// Un thunk: una función que se despacha como si fuera una acción. El
  middleware
// redux-thunk la ejecuta pasándole dispatch y getState, así puede
  hacer cosas async.
export function cargarPostulaciones(falla: boolean):
  ThunkAction<Promise<void>, State, unknown, Action> {
  return async (dispatch, getState) => {
    if (getState().status === 'loading') return // ya hay una carga en
      curso
    dispatch(cargaIniciada())
    try {
      dispatch(cargaExitosa(await api.listar({ falla })))
    } catch (e) {
      dispatch(cargaFallida(e instanceof Error ? e.message : 'Error'))
    }
  }
}

// Un middleware se mete entre el dispatch y el reducer: ve pasar cada
  acción.
// Este anota cada una (como hace Redux DevTools, pero en la demo).
export function crearLogger(anotar: (linea: string) => void):
  Middleware<object, State> {
  return (api) => (next) => (action) => {
    const antes = api.getState().status
    const resultado = next(action) // pasa la acción al siguiente
      middleware (o al reducer)
    const tipo = typeof action === 'function' ? '(thunk)' : (action as
      Action).type
    anotar(`${tipo}  ·  status: ${antes} → ${api.getState().status}`)
    return resultado
  }
}
```

```
export function crearStore(anotar: (linea: string) => void) {
  // El orden importa: thunk primero, así el logger solo ve acciones
    (objetos).
  return createStore(reducer, applyMiddleware(thunk,
    crearLogger(anotar)))
}

export type AppDispatch = ThunkDispatch<State, unknown, Action>
```

Tocá **Cargar** y mirá el log: el logger ve pasar `cargaIniciada`, y 700 ms después `cargaExitosa` (o `cargaFallida`). El thunk en sí no llega al logger porque `thunk` va primero en la cadena y lo "consume".

Mirá la pestaña `store.ts`: así se escribía Redux en muchos proyectos. Para **una** operación async hay tres constantes, tres action creators, tres `case` en el reducer y el thunk. Multiplicalo por cada endpoint de la app.

El problema del boilerplate

Por cada feature, en Redux clásico...	Con Redux Toolkit
Constantes de tipos de acción	Las genera `createSlice`
Action creators a mano	Los genera `createSlice`
Reducer con `switch` y spreads anidados	Funciones que "mutan" un borrador (Immer)
Thunk con pending/success/error a mano	`createAsyncThunk`
Configurar thunk, DevTools y chequeos	`configureStore` lo trae todo

Por eso nació Redux Toolkit, que vemos en la parte siguiente. Pero entender esto te sirve para leer código legacy, que es justamente donde más Redux clásico vas a encontrar.

ℹ️ Otros middlewares

`redux-saga` (parte 3) y `redux-observable` (con RxJS) son middlewares para efectos más complejos. `redux-logger` hace lo del ejemplo. Y Redux Toolkit trae su propio **listener middleware** para reaccionar a acciones (lo vemos en la comparación).

🎤 En una entrevista

"¿Dónde hacés un fetch en Redux?" → "No en el reducer, que tiene que ser puro. En un middleware: lo más común es un thunk (o createAsyncThunk en Redux Toolkit) que despacha pending, success o error. Si los datos son del servidor, prefiero RTK Query o TanStack Query, que además manejan la caché."

Parte 2 · Redux Toolkit

La forma oficial de escribir Redux: slices, thunks generados y RTK Query para los datos del servidor.

Capítulo 5

configureStore y createSlice

El mismo Redux con mucho menos código: slices, Immer y tipos que salen del store.

Redux Toolkit (RTK) es la forma oficial y recomendada de escribir Redux desde 2019. No es otra librería: es Redux, con funciones que eliminan el boilerplate y traen buenas prácticas por defecto.

× Redux clásico

```
const AVANZADA = 'postulaciones/avanzada'
const avanzada = (id: number) => ({ type: AVANZADA, payload: id })

function reducer(state = inicial, action) {
  switch (action.type) {
    case AVANZADA:
      return {
        ...state,
        items: state.items.map((p) =>
          p.id === action.payload ? { ...p, etapa: siguiente[p.etapa]
            } : p,
        ),
      }
    default:
      return state
  }
}
```

✓ Redux Toolkit

```
const slice = createSlice({
```

```
  name: 'postulaciones',
  initialState: inicial,
  reducers: {
    avanzada(state, action: PayloadAction<number>) {
      const p = state.items.find((p) => p.id === action.payload)
      if (p) p.etapa = siguiente[p.etapa]
    },
  },
})

export const { avanzada } = slice.actions
export default slice.reducer
```

createSlice

Un **slice** es una porción del estado con su reducer y sus acciones. `createSlice` recibe un nombre, el estado inicial y los "case reducers", y genera:

- `slice.reducer`: el reducer para el store.
- `slice.actions`: un action creator por cada reducer, con el `type` armado como `nombre/reducer` (`postulaciones/avanzada`).
- `slice.selectors`: los selectores que declares, adaptados para recibir el estado global.

> **⚠️ ¿Mutar? Sí, pero es un borrador**
>
> Adentro de `createSlice` podés escribir `state.filtro = 'x'` o `state.items.push(p)`. No estás mutando el estado real: RTK usa **Immer**, que te da un borrador, registra los cambios y arma un estado nuevo inmutable. Dos reglas: o mutás el borrador, o devolvés un valor nuevo, **nunca las dos cosas**. Y esto solo vale adentro de `createSlice` / `createReducer`: en un componente, sigue sin poder mutarse.

configureStore y los tipos

src/demos/estado/toolkit/store.ts

```ts
import { configureStore } from '@reduxjs/toolkit'
import { useDispatch, useSelector } from 'react-redux'
import postulacionesSlice from './postulacionesSlice'

// configureStore ya trae thunk, Redux DevTools y chequeos de
  mutaciones en desarrollo.
export function crearStore() {
  return configureStore({
    reducer: {
      [postulacionesSlice.reducerPath]: postulacionesSlice.reducer, //
        reducerPath es 'postulaciones'
    },
  })
}

// Los tipos salen del store: no hay que escribirlos a mano.
export type AppStore = ReturnType<typeof crearStore>
export type RootState = ReturnType<AppStore['getState']>
export type AppDispatch = AppStore['dispatch']

export const useAppSelector = useSelector.withTypes<RootState>()
export const useAppDispatch = useDispatch.withTypes<AppDispatch>()
```

`configureStore` arma el store con buenos defaults: combina los reducers, agrega el middleware de thunks, conecta las Redux DevTools y, en desarrollo, chequea que no mutes el estado ni guardes valores no serializables (como un `Date` o una función).

Los tipos `RootState` y `AppDispatch` **salen del store**, no se escriben a mano. Con ellos se crean los hooks tipados una sola vez.

> 💡 **Un store por request (Next.js)**
>
> En la demo el store se crea con una función (`crearStore`) y no como una constante global. Es el patrón que pide Next.js con el App Router: un store por request en el servidor, para no mezclar datos de distintos usuarios. En una app solo de cliente (Vite), una constante global también está bien.

La demo completa

DEMO **Slice, selectores memorizados y thunks con createAsyncThunk**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/estado/ReduxToolkit.tsx`

```
import { useState } from 'react'
import { Provider } from 'react-redux'
import {
  agregarPostulacion,
  avanzada,
  cargarPostulaciones,
  filtroCambiado,
  rechazada,
  selectError,
  selectFiltradas,
  selectFiltro,
  selectStatus,
} from './toolkit/postulacionesSlice'
import { crearStore, useAppDispatch, useAppSelector } from
  './toolkit/store'

export default function ReduxToolkit() {
  const [store] = useState(crearStore)
  return (
    <Provider store={store}>
      <Postulaciones />
    </Provider>
  )
}
```

```
function Postulaciones() {
  const dispatch = useAppDispatch()
  const status = useAppSelector(selectStatus)
  const error = useAppSelector(selectError)
  const filtro = useAppSelector(selectFiltro)
  const filtradas = useAppSelector(selectFiltradas)
  const [empresa, setEmpresa] = useState('')

  return (
    <div className="stack">
      <div className="row">
        <button onClick={() =>
          dispatch(cargarPostulaciones(false))}>Cargar</button>
        <button onClick={() =>
          dispatch(cargarPostulaciones(true))}>Cargar (con error)
          </button>
        <span className="pill">status: {status}</span>
        <input value={filtro} onChange={(e) =>
          dispatch(filtroCambiado(e.target.value))}
          placeholder="Filtrar" />
      </div>
      {error && <p className="error-text">{error}</p>}
      <ul>
        {filtradas.map((p) => (
          <li key={p.id}>
            {p.empresa} · {p.etapa} <button onClick={() =>
              dispatch(avanzada(p.id))}>avanzar</button>{' '}
            <button onClick={() =>
              dispatch(rechazada(p.id))}>x</button>
          </li>
        ))}
      </ul>
      <form
        className="row"
        onSubmit={async (e) => {
          e.preventDefault()
          if (!empresa.trim()) return
          // unwrap() devuelve el payload o tira el error: útil para
            reaccionar en el componente.
          await dispatch(agregarPostulacion(empresa.trim())).unwrap()
          setEmpresa('')
        }}
      >
        <input value={empresa} onChange={(e) =>
          setEmpresa(e.target.value)} placeholder="Nueva empresa" />
        <button type="submit">Agregar</button>
      </form>
    </div>
  )
```

```
}
```

src/demos/estado/toolkit/postulacionesSlice.ts

```
import { createAsyncThunk, createSelector, createSlice, type
  PayloadAction } from '@reduxjs/toolkit'
import { api, siguienteEtapa, type Postulacion } from '../api'

type PostulacionesState = {
  items: Postulacion[]
  filtro: string
  status: 'idle' | 'loading' | 'succeeded' | 'failed'
  error: string | null
}

const initialState: PostulacionesState = { items: [], filtro: '',
  status: 'idle', error: null }

// createAsyncThunk genera el thunk y las tres acciones: pending,
  fulfilled y rejected.
export const cargarPostulaciones = createAsyncThunk(
  'postulaciones/cargar',
  async (falla: boolean) => api.listar({ falla }), // lo que devuelve
    va en action.payload
  {
    // Si ya está cargando, ni siquiera arranca (no despacha pending).
    condition: (_falla, { getState }) =>
      (getState() as { postulaciones: PostulacionesState
        }).postulaciones.status !== 'loading',
  },
)

export const agregarPostulacion =
  createAsyncThunk('postulaciones/agregar', (empresa: string) =>
  api.crear(empresa))

// createSlice: el reducer y los action creators en un solo lugar.
const postulacionesSlice = createSlice({
  name: 'postulaciones',
  initialState,
  reducers: {
    // Gracias a Immer, se "muta" un borrador y RTK arma el estado
      nuevo inmutable.
    avanzada(state, action: PayloadAction<number>) {
      const postulacion = state.items.find((p) => p.id ===
        action.payload)
      if (postulacion) postulacion.etapa =
        siguienteEtapa[postulacion.etapa]
    },
    rechazada(state, action: PayloadAction<number>) {
```

```
      const postulacion = state.items.find((p) => p.id ===
        action.payload)
      if (postulacion) postulacion.etapa = 'rechazado'
    },
    filtroCambiado(state, action: PayloadAction<string>) {
      state.filtro = action.payload
    },
  },
  // Las acciones que NO genera este slice (como las del thunk) se
    manejan acá.
  extraReducers: (builder) => {
    builder
      .addCase(cargarPostulaciones.pending, (state) => {
        state.status = 'loading'
        state.error = null
      })
      .addCase(cargarPostulaciones.fulfilled, (state, action) => {
        state.status = 'succeeded'
        state.items = action.payload
      })
      .addCase(cargarPostulaciones.rejected, (state, action) => {
        state.status = 'failed'
        state.error = action.error.message ?? 'Error'
      })
      .addCase(agregarPostulacion.fulfilled, (state, action) => {
        state.items.push(action.payload)
      })
  },
  // Selectores escritos sobre el estado del slice. RTK los adapta
    para recibir el estado global.
  selectors: {
    selectStatus: (state) => state.status,
    selectError: (state) => state.error,
    selectFiltro: (state) => state.filtro,
    // createSelector memoriza: devuelve el mismo array mientras items
      y filtro no cambien.
    selectFiltradas: createSelector(
      [(state: PostulacionesState) => state.items, (state:
        PostulacionesState) => state.filtro],
      (items, filtro) => items.filter((p) =>
        p.empresa.toLowerCase().includes(filtro.toLowerCase())),
    ),
  },
})

export const { avanzada, rechazada, filtroCambiado } =
  postulacionesSlice.actions
export const { selectStatus, selectError, selectFiltro,
  selectFiltradas } = postulacionesSlice.selectors
export default postulacionesSlice
```

src/demos/estado/toolkit/store.ts

```ts
import { configureStore } from '@reduxjs/toolkit'
import { useDispatch, useSelector } from 'react-redux'
import postulacionesSlice from './postulacionesSlice'

// configureStore ya trae thunk, Redux DevTools y chequeos de
  mutaciones en desarrollo.
export function crearStore() {
  return configureStore({
    reducer: {
      [postulacionesSlice.reducerPath]: postulacionesSlice.reducer, //
        reducerPath es 'postulaciones'
    },
  })
}

// Los tipos salen del store: no hay que escribirlos a mano.
export type AppStore = ReturnType<typeof crearStore>
export type RootState = ReturnType<AppStore['getState']>
export type AppDispatch = AppStore['dispatch']

export const useAppSelector = useSelector.withTypes<RootState>()
export const useAppDispatch = useDispatch.withTypes<AppDispatch>()
```

Cargá la lista, filtrá, avanzá y rechazá postulaciones, y agregá una nueva. Mirá la pestaña del slice: todo lo que en Redux clásico estaba repartido en cuatro lugares, acá está en un archivo. Lo async (`createAsyncThunk`) lo vemos en el capítulo siguiente.

> **🎤 En una entrevista**
>
> "¿Qué es Redux Toolkit y por qué usarlo?" → "Es la forma oficial de escribir Redux. createSlice genera acciones y reducer a partir de funciones que mutan un borrador con Immer, configureStore trae thunk, DevTools y chequeos, createAsyncThunk maneja pending, fulfilled y rejected, y RTK Query resuelve el fetch y la caché. Es el mismo Redux con mucho menos código."

Capítulo 6

createAsyncThunk y compañía

pending, fulfilled y rejected generados, unwrap, entity adapter y listener middleware.

Para lo async, Redux Toolkit trae `createAsyncThunk`: le das un nombre y una función async, y genera el thunk **y** las tres acciones del ciclo de vida.

src/demos/estado/toolkit/postulacionesSlice.ts

```ts
import { createAsyncThunk, createSelector, createSlice, type
  PayloadAction } from '@reduxjs/toolkit'
import { api, siguienteEtapa, type Postulacion } from '../api'

type PostulacionesState = {
  items: Postulacion[]
  filtro: string
  status: 'idle' | 'loading' | 'succeeded' | 'failed'
  error: string | null
}

const initialState: PostulacionesState = { items: [], filtro: '',
  status: 'idle', error: null }

// createAsyncThunk genera el thunk y las tres acciones: pending,
  fulfilled y rejected.
export const cargarPostulaciones = createAsyncThunk(
  'postulaciones/cargar',
  async (falla: boolean) => api.listar({ falla }), // lo que devuelve
    va en action.payload
  {
    // Si ya está cargando, ni siquiera arranca (no despacha pending).
    condition: (_falla, { getState }) =>
      (getState() as { postulaciones: PostulacionesState
        }).postulaciones.status !== 'loading',
  },
)
```

```ts
export const agregarPostulacion =
  createAsyncThunk('postulaciones/agregar', (empresa: string) =>
  api.crear(empresa))

// createSlice: el reducer y los action creators en un solo lugar.
const postulacionesSlice = createSlice({
  name: 'postulaciones',
  initialState,
  reducers: {
    // Gracias a Immer, se "muta" un borrador y RTK arma el estado
      nuevo inmutable.
    avanzada(state, action: PayloadAction<number>) {
      const postulacion = state.items.find((p) => p.id ===
        action.payload)
      if (postulacion) postulacion.etapa =
        siguienteEtapa[postulacion.etapa]
    },
    rechazada(state, action: PayloadAction<number>) {
      const postulacion = state.items.find((p) => p.id ===
        action.payload)
      if (postulacion) postulacion.etapa = 'rechazado'
    },
    filtroCambiado(state, action: PayloadAction<string>) {
      state.filtro = action.payload
    },
  },
  // Las acciones que NO genera este slice (como las del thunk) se
    manejan acá.
  extraReducers: (builder) => {
    builder
      .addCase(cargarPostulaciones.pending, (state) => {
        state.status = 'loading'
        state.error = null
      })
      .addCase(cargarPostulaciones.fulfilled, (state, action) => {
        state.status = 'succeeded'
        state.items = action.payload
      })
      .addCase(cargarPostulaciones.rejected, (state, action) => {
        state.status = 'failed'
        state.error = action.error.message ?? 'Error'
      })
      .addCase(agregarPostulacion.fulfilled, (state, action) => {
        state.items.push(action.payload)
      })
  },
  // Selectores escritos sobre el estado del slice. RTK los adapta
    para recibir el estado global.
  selectors: {
    selectStatus: (state) => state.status,
    selectError: (state) => state.error,
```

```
    selectFiltro: (state) => state.filtro,
    // createSelector memoriza: devuelve el mismo array mientras items
      y filtro no cambien.
    selectFiltradas: createSelector(
      [(state: PostulacionesState) => state.items, (state:
        PostulacionesState) => state.filtro],
      (items, filtro) => items.filter((p) =>
        p.empresa.toLowerCase().includes(filtro.toLowerCase())),
    ),
  },
})

export const { avanzada, rechazada, filtroCambiado } =
  postulacionesSlice.actions
export const { selectStatus, selectError, selectFiltro,
  selectFiltradas } = postulacionesSlice.selectors
export default postulacionesSlice
```

createAsyncThunk

```
const cargarPostulaciones = createAsyncThunk('postulaciones/cargar',
  async (falla: boolean) => {
  return api.listar({ falla }) // lo que devuelve es el payload de
    fulfilled
})
```

Acción generada	Cuándo se despacha	Qué trae
`cargarPostulaciones.pending`	Al empezar	`meta.arg` (el argumento)
`cargarPostulaciones.fulfilled`	Si la promesa se resuelve	`payload` (lo que devolviste)
`cargarPostulaciones.rejected`	Si tira error	`error` (serializado) o `payload` si usás `rejectWithValue`

Las tres se manejan en `extraReducers` con el `builder`, porque no las "genera" el slice. El builder con `addCase` es la única forma desde RTK 2 (la sintaxis de objeto se eliminó) y tiene inferencia de tipos completa.

Opciones útiles

- `condition`: si devuelve `false`, el thunk no arranca. En la demo evita dos cargas al mismo tiempo.
- `rejectWithValue`: para devolver un error con forma propia (por ejemplo, los errores de validación del backend).
- `unwrap()`: `await dispatch(thunk()).unwrap()` devuelve el payload o tira el error. Sirve para reaccionar en el componente (cerrar un modal, mostrar un toast).
- `getState` y `dispatch`: el segundo parámetro de la función (`thunkAPI`) los trae, por si necesitás leer el estado o despachar otras acciones.
- `signal`: un `AbortSignal` para cancelar el `fetch` si el thunk se aborta.

Otras herramientas de RTK

Herramienta	Para qué
`createEntityAdapter`	Guardar colecciones normalizadas (`ids` + `entities`) con reducers y selectores listos (`addOne`, `updateOne`, `selectAll`...).
`createListenerMiddleware`	Reaccionar a acciones con lógica async ("cuando pase X, hacé Y"). Es la alternativa liviana a Redux Saga.
`combineSlices`	Combinar slices y cargarlos de forma diferida (code splitting).
`createSelector`	Reselect, reexportado.

> ⚠️ **¿Thunks para todo lo del servidor?**
>
> Si el thunk solo trae datos del backend para mostrarlos, estás escribiendo una caché a mano: `status`, `error`, cuándo volver a pedir, qué pasa si dos componentes piden lo mismo... Para eso está **RTK Query**, el capítulo siguiente. Los thunks quedan para lógica que no es "traer datos": un flujo de checkout, sincronizar con `localStorage`, coordinar varias acciones.

🎤 En una entrevista

"¿Qué genera createAsyncThunk?" → "Un thunk y tres action creators: pending, fulfilled y rejected, que manejo en extraReducers con el builder. Lo que devuelve la función async es el payload de fulfilled y un error se convierte en rejected. Con unwrap puedo esperar el resultado en el componente."

Capítulo 7

RTK Query

Fetching y caché: queries, mutations, tags e invalidación. Comparado con TanStack Query.

RTK Query es la capa de **fetching y caché** de Redux Toolkit. Vos describís los endpoints y genera los hooks, el estado de carga, la caché compartida, la deduplicación de pedidos y la invalidación. Es la respuesta de Redux a TanStack Query, y viene incluida en `@reduxjs/toolkit`.

DEMO **Queries, mutations y caché compartida con RTK Query**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/estado/RtkQuery.tsx`

```
import { configureStore } from '@reduxjs/toolkit'
import { useState } from 'react'
import { Provider } from 'react-redux'
import { postulacionesApi, reiniciarDatos, useAvanzarMutation,
  useListarQuery } from './rtk-query/api'

function crearStore() {
  reiniciarDatos()
  return configureStore({
    reducer: { [postulacionesApi.reducerPath]:
      postulacionesApi.reducer },
    // El middleware de RTK Query maneja la caché, los reintentos y la
      invalidación.
    middleware: (getDefault) =>
      getDefault().concat(postulacionesApi.middleware),
  })
}

export default function RtkQuery() {
```

```
  const [store] = useState(crearStore)
  const [mostrarSegunda, setMostrarSegunda] = useState(false)
  return (
    <Provider store={store}>
      <div className="stack">
        <div className="grid-2">
          <Lista titulo="Componente A" />
          {mostrarSegunda ? (
            <Lista titulo="Componente B (comparte la caché)" />
          ) : (
            <button onClick={() => setMostrarSegunda(true)}>Montar
              otro componente que pide lo mismo</button>
          )}
        </div>
      </div>
    </Provider>
  )
}

function Lista({ titulo }: { titulo: string }) {
  // data, isLoading (la primera vez), isFetching (cualquier pedido),
    error y refetch.
  const { data = [], isLoading, isFetching, error, refetch } =
    useListarQuery()
  const [avanzar, { isLoading: avanzando }] = useAvanzarMutation()

  return (
    <div className="card stack">
      <div className="row">
        <strong style={{ flex: 1 }}>{titulo}</strong>
        <span className="pill">{isLoading ? 'cargando' : isFetching ?
          'actualizando' : 'en caché'}</span>
        <button onClick={refetch}>refetch</button>
      </div>
      {error && <p className="error-text">{'message' in error ?
        error.message : 'Error'}</p>}
      <ul style={{ opacity: isFetching ? 0.5 : 1 }}>
        {data.map((p) => (
          <li key={p.id}>
            {p.empresa} · {p.etapa}{' '}
            <button disabled={avanzando || p.etapa === 'oferta'}
              onClick={() => avanzar(p.id)}>
              avanzar
            </button>
          </li>
        ))}
      </ul>
    </div>
  )
}
```

src/demos/estado/rtk-query/api.ts

```ts
import { createApi, fakeBaseQuery } from
  '@reduxjs/toolkit/query/react'
import { api as servidor, siguienteEtapa, type Postulacion } from
  '../api'

// En un proyecto real usarías fetchBaseQuery({ baseUrl: '/api' }) y
  cada endpoint sería
// query: () => 'postulaciones'. Acá usamos queryFn para llamar a la
  API falsa de la guía.
let datos: Postulacion[] | null = null

export const postulacionesApi = createApi({
  reducerPath: 'postulacionesApi',
  baseQuery: fakeBaseQuery<{ message: string }>(),
  tagTypes: ['Postulacion'],
  endpoints: (build) => ({
    // Una query: lee datos y los guarda en caché.
    listar: build.query<Postulacion[], void>({
      async queryFn() {
        try {
          datos ??= await servidor.listar()
          await servidor.ofertasNuevas() // un poco de demora en cada
            pedido
          return { data: datos.map((p) => ({ ...p })) }
        } catch (e) {
          return { error: { message: e instanceof Error ? e.message :
            'Error' } }
        }
      },
      providesTags: ['Postulacion'], // "esta caché tiene
        postulaciones"
    }),
    // Una mutation: cambia datos en el servidor.
    avanzar: build.mutation<Postulacion, number>({
      async queryFn(id) {
        await servidor.ofertasNuevas()
        const postulacion = datos?.find((p) => p.id === id)
        if (!postulacion) return { error: { message: 'No existe' } }
        postulacion.etapa = siguienteEtapa[postulacion.etapa]
        return { data: { ...postulacion } }
      },
      invalidatesTags: ['Postulacion'], // al terminar, las queries
        con ese tag se vuelven a pedir
    }),
  }),
})
```

```
// RTK Query genera un hook por endpoint: use + Nombre +
  Query/Mutation.
export const { useListarQuery, useAvanzarMutation } = postulacionesApi

export function reiniciarDatos() {
  datos = null
}
```

Probá esto:

1. Montá el segundo componente: aparece **al instante**, sin pedir de nuevo. Los dos leen la misma entrada de la caché.
2. Tocá **avanzar** en cualquiera: la mutation invalida el tag `Postulacion` y la lista se vuelve a pedir sola. Los dos componentes se actualizan.
3. **refetch** fuerza un pedido nuevo: `isFetching` pasa a `true`, pero `isLoading` no (ya había datos).

Cómo se define una API

```
export const ofertasApi = createApi({
  reducerPath: 'ofertasApi',
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  tagTypes: ['Oferta'],
  endpoints: (build) => ({
    listar: build.query<Oferta[], void>({
      query: () => 'ofertas',
      providesTags: ['Oferta'],
    }),
    crear: build.mutation<Oferta, NuevaOferta>({
      query: (body) => ({ url: 'ofertas', method: 'POST', body }),
      invalidatesTags: ['Oferta'],
    }),
  }),
})

export const { useListarQuery, useCrearMutation } = ofertasApi
```

La demo usa `queryFn` en vez de `query` para llamar a la API falsa de la guía, pero en un proyecto real es así de corto. Después se suma al

store: el reducer en `reducerPath` y el middleware con `.concat(ofertasApi.middleware)`.

Lo que te da cada hook

Query (`useListarQuery(arg)`)	Mutation (`useCrearMutation()`)
`data`, `error`	Devuelve `[disparar, { isLoading, error, data }]`
`isLoading`: primera carga, sin datos	`disparar(arg)` devuelve una promesa con `.unwrap()`
`isFetching`: cualquier pedido en curso	Invalida tags al terminar
`refetch()`	
Opciones: `skip`, `pollingInterval`, `refetchOnFocus`	

Tags: cuándo volver a pedir

Las queries **proveen** tags y las mutations los **invalidan**. Cuando una mutation invalida `'Oferta'`, toda query que provea `'Oferta'` y esté en uso se vuelve a pedir. Con ids (`{ type: 'Oferta', id: 3 }`) se puede invalidar solo un elemento.

RTK Query o TanStack Query

	RTK Query	TanStack Query
Si ya usás Redux	Ideal: misma store y DevTools	Funciona igual, separado
Sin Redux	Trae Redux como dependencia	Más liviano
Definición	Endpoints centralizados en `createApi`	Hooks con `queryKey` donde los uses
Caché	Por endpoint + argumento, tags	Por `queryKey`, invalidación por key

Las dos resuelven lo mismo. En un proyecto con Redux, RTK Query; en uno sin Redux (o con Zustand), TanStack Query.

🎤 En una entrevista

"¿Por qué no guardar la respuesta de la API en un slice con un thunk?" → "Porque estaría reescribiendo una caché: loading, error, deduplicar pedidos, invalidar al mutar, refetch al volver a la pestaña. RTK Query (o TanStack Query) ya lo resuelve: defino endpoints, uso los hooks y con tags invalido lo que cambió."

Parte 3 · Redux Saga

Efectos async con generadores: cancelación, carreras, procesos de fondo y tests sin mocks.

Capítulo 8

Generadores y efectos

call, put, take, takeEvery y takeLatest, con un buscador con debounce y cancelación.

Redux Saga es un middleware para manejar efectos secundarios con **funciones generadoras**. Mientras un thunk es "una función async que despacha cosas", una saga es un **proceso** que escucha acciones y decide qué hacer: puede esperar, cancelarse, correr en paralelo o competir con otras.

Lo vas a encontrar en muchos proyectos grandes de 2017 a 2021 (bancos, fintech, e-commerce), justamente el tipo de código legacy que suele pedirse en entrevistas.

Repaso: generadores

Una función con `function*` es un **generador**: se puede pausar en cada `yield` y alguien de afuera decide cuándo sigue y con qué valor.

```
function* contar() {
  const a: number = yield 'primero' // se pausa acá
  const b: number = yield 'segundo'
  return a + b
}

const it = contar()
it.next() // { value: 'primero', done: false }
```

```
it.next(1) // a = 1 → { value: 'segundo', done: false }
it.next(2) // b = 2 → { value: 3, done: true }
```

El middleware de Redux Saga es ese "alguien de afuera": la saga hace `yield` de un **efecto** (un objeto que describe qué hacer), el middleware lo ejecuta y le devuelve el resultado.

Los efectos básicos

Efecto	Qué hace	Bloquea
`call(fn, ...args)`	Llama a una función (si devuelve una promesa, espera el resultado)	Sí
`put(acción)`	Despacha una acción	No
`select(selector)`	Lee el estado	No
`take('tipo')`	Espera a que se despache una acción de ese tipo	Sí
`delay(ms)`	Espera	Sí
`fork(saga)`	Arranca otra saga en paralelo	No
`takeEvery('tipo', saga)`	Arranca `saga` por **cada** acción de ese tipo	No
`takeLatest('tipo', saga)`	Igual, pero **cancela** la anterior si sigue corriendo	No

Un buscador con debounce y cancelación

DEMO **takeLatest + delay: debounce y cancelación en cuatro líneas**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/estado/SagaBusqueda.tsx`

```
import { useState } from 'react'
import { Provider, useDispatch, useSelector } from 'react-redux'
```

```
import { crearStore, textoCambiado, type RootState } from
  './saga/busqueda'

export default function SagaBusqueda() {
  const [store] = useState(crearStore)
  return (
    <Provider store={store}>
      <Buscador />
    </Provider>
  )
}

function Buscador() {
  const { texto, resultados, buscando, log } = useSelector((state:
    RootState) => state.busqueda)
  const dispatch = useDispatch()
  return (
    <div className="stack">
      <div className="row">
        <input
          value={texto}
          onChange={(e) => dispatch(textoCambiado(e.target.value))}
          placeholder="Buscá una empresa (probá escribir rápido)"
          style={{ flex: 1 }}
        />
        {buscando && <span className="pill">buscando…</span>}
      </div>
      <div className="grid-2">
        <ul>
          {resultados.map((r) => (
            <li key={r}>{r}</li>
          ))}
        </ul>
        <ol className="log">
          {log.length ? log.slice(-8).map((l, i) => <li key={i}>{l}
            </li>) : <li>(escribí algo)</li>}
        </ol>
      </div>
    </div>
  )
}
```

src/demos/estado/saga/busqueda.ts

```
import { configureStore, createSlice, type PayloadAction } from
  '@reduxjs/toolkit'
import createSagaMiddleware from 'redux-saga'
import { call, cancelled, delay, put, takeLatest } from 'redux-
  saga/effects'
import { api } from '../api'
```

```
// El slice solo guarda datos: la lógica async vive en la saga.
const busquedaSlice = createSlice({
  name: 'busqueda',
  initialState: { texto: '', resultados: [] as string[], buscando:
    false, log: [] as string[] },
  reducers: {
    textoCambiado(state, action: PayloadAction<string>) {
      state.texto = action.payload
    },
    busquedaIniciada(state, action: PayloadAction<string>) {
      state.buscando = true
      state.log.push(`▶ buscando "${action.payload}"`)
    },
    resultadosRecibidos(state, action: PayloadAction<{ texto: string;
      resultados: string[] }>) {
      state.buscando = false
      state.resultados = action.payload.resultados
      const n = action.payload.resultados.length
      state.log.push(`✓ ${n} resultado${n === 1 ? '' : 's'} para
        "${action.payload.texto}"`)
    },
    busquedaCancelada(state, action: PayloadAction<string>) {
      state.log.push(`× cancelada "${action.payload}" (llegó otra
        tecla)`)
    },
  },
})

export const { textoCambiado } = busquedaSlice.actions
const { busquedaIniciada, resultadosRecibidos, busquedaCancelada } =
  busquedaSlice.actions

// Una saga es una función generadora. Cada yield le pide algo al
  middleware (un "efecto")
// y el middleware lo ejecuta: llamar a una función, despachar una
  acción, esperar…
function* buscar(action: PayloadAction<string>) {
  const texto = action.payload.trim()
  if (!texto) return
  try {
    yield delay(300) // debounce: si llega otra tecla antes,
      takeLatest cancela esta saga acá
    yield put(busquedaIniciada(texto))
    const resultados: string[] = yield call(api.buscarEmpresas, texto)
    yield put(resultadosRecibidos({ texto, resultados }))
  } finally {
    // finally corre siempre; cancelled() dice si fue porque la
      cancelaron.
    const fueCancelada: boolean = yield cancelled()
    if (fueCancelada) yield put(busquedaCancelada(texto))
```

```
  }
}

// La saga raíz: por cada textoCambiado arranca buscar, cancelando la
  anterior si sigue viva.
function* rootSaga() {
  yield takeLatest(textoCambiado.type, buscar)
}

export function crearStore() {
  const sagaMiddleware = createSagaMiddleware()
  const store = configureStore({
    reducer: { busqueda: busquedaSlice.reducer },
    middleware: (getDefault) => getDefault({ thunk: false
      }).concat(sagaMiddleware),
  })
  sagaMiddleware.run(rootSaga)
  return store
}

export type RootState = ReturnType<ReturnType<typeof crearStore>
  ['getState']>
```

Escribí rápido en el buscador y mirá el log: cada tecla arranca una saga nueva, y `takeLatest` **cancela** la anterior. Si la cancelación llega durante el `delay(300)`, nunca se busca: eso es un debounce. Si llega durante el `call` a la API, la respuesta vieja se descarta y no pisa a la nueva (el famoso problema de las *race conditions* del capítulo de `useEffect`).

En la pestaña `busqueda.ts`:

- El slice de RTK solo guarda datos. La lógica async vive en la saga `buscar`.
- `try/finally` con `cancelled()` permite reaccionar a la cancelación (limpiar, anotar, abortar).
- El store se arma con `configureStore` y el middleware de Saga, que después se arranca con `sagaMiddleware.run(rootSaga)`.

💡 Los efectos son datos

`call(api.buscarEmpresas, texto)` no llama a la API: devuelve un objeto `{ CALL: { fn, args } }`. Eso hace que las sagas sean muy fáciles de testear: recorrés el generador con `next()` y comparás objetos, sin mockear `fetch`. Lo vemos en el capítulo siguiente.

🎤 En una entrevista

"¿Qué diferencia hay entre takeEvery y takeLatest?" → "Los dos arrancan una saga por cada acción de un tipo. takeEvery deja correr todas en paralelo; takeLatest cancela la anterior si sigue viva, así solo cuenta la última. takeLatest es ideal para búsquedas o cargas donde una respuesta vieja no debe pisar a la nueva."

Capítulo 9

Patrones: race, fork y cancel

Timeout, polling, manejo de errores, testing y cuándo usar Saga hoy.

Saga brilla cuando el flujo async es **complejo**: varias cosas que compiten, procesos que corren de fondo y hay que poder cortar, pasos que esperan a que el usuario haga algo. Estos son los patrones que más vas a ver.

DEMO **race para un timeout y fork/cancel para un polling**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/estado/SagaPatrones.tsx`

```
import { useEffect, useState } from 'react'
import { Provider, useDispatch, useSelector } from 'react-redux'
import { crearStore, pedidoConTimeout, pollingDetenido,
  pollingIniciado, type RootState } from './saga/patrones'

export default function SagaPatrones() {
  const [store] = useState(crearStore)
  // Al desmontar la demo, cortamos el polling (si no, seguiría
    pidiendo para siempre).
  useEffect(() => () => void store.dispatch(pollingDetenido()),
    [store])
  return (
    <Provider store={store}>
      <Panel />
    </Provider>
  )
}

function Panel() {
```

```
  const { log, ofertas, polling } = useSelector((state: RootState) =>
    state.patrones)
  const dispatch = useDispatch()
  return (
    <div className="stack">
      <div className="row">
        <button onClick={() => dispatch(pedidoConTimeout())}>Pedido
          con timeout (race)</button>
        <button onClick={() => dispatch(polling ? pollingDetenido() :
          pollingIniciado())}>
          {polling ? 'Detener polling' : 'Iniciar polling'}
        </button>
        <span className="pill">ofertas nuevas: {ofertas ?? '–'}</span>
      </div>
      <ol className="log">
        {log.length ? log.slice(-8).map((l, i) => <li key={i}>{l}
          </li>) : <li>(tocá un botón)</li>}
      </ol>
    </div>
  )
}
```

src/demos/estado/saga/patrones.ts

```
import { configureStore, createAction, createSlice, type PayloadAction
  } from '@reduxjs/toolkit'
import createSagaMiddleware, { type Task } from 'redux-saga'
import { call, cancel, delay, fork, put, race, take, takeEvery } from
  'redux-saga/effects'
import { api } from '../api'

const patronesSlice = createSlice({
  name: 'patrones',
  initialState: { log: [] as string[], ofertas: null as number | null,
    polling: false },
  reducers: {
    anotado(state, action: PayloadAction<string>) {
      state.log.push(action.payload)
    },
    ofertasActualizadas(state, action: PayloadAction<number>) {
      state.ofertas = action.payload
    },
    pollingCambiado(state, action: PayloadAction<boolean>) {
      state.polling = action.payload
    },
  },
})
const { anotado, ofertasActualizadas, pollingCambiado } =
  patronesSlice.actions
```

```ts
// Acciones que solo escuchan las sagas (ningún reducer las maneja).
export const pedidoConTimeout =
  createAction('patrones/pedidoConTimeout')
export const pollingIniciado =
  createAction('patrones/pollingIniciado')
export const pollingDetenido =
  createAction('patrones/pollingDetenido')

// race: corre varios efectos y se queda con el primero que termina
  (los demás se cancelan).
function* pedirConTimeout() {
  yield put(anotado('▶ pidiendo… (timeout de 2 s)'))
  const { respuesta, timeout }: { respuesta?: string; timeout?: true }
    = yield race({
    respuesta: call(api.lenta),
    timeout: delay(2000, true),
  })
  yield put(anotado(timeout ? '⏱ timeout: el pedido se canceló' : `✓
    ${respuesta}`))
}

// Polling: un loop infinito con delay. Corre en paralelo (fork) hasta
  que lo cancelan.
function* polling() {
  while (true) {
    const cantidad: number = yield call(api.ofertasNuevas)
    yield put(ofertasActualizadas(cantidad))
    yield delay(1500)
  }
}

function* controlarPolling() {
  while (true) {
    yield take(pollingIniciado.type) // espera a que alguien despache
      la acción
    yield put(pollingCambiado(true))
    const tarea: Task = yield fork(polling) // arranca sin bloquear
    yield take(pollingDetenido.type)
    yield cancel(tarea) // cancela el loop, aunque esté en medio de un
      call o un delay
    yield put(pollingCambiado(false))
  }
}

function* rootSaga() {
  yield fork(controlarPolling)
  yield takeEvery(pedidoConTimeout.type, pedirConTimeout)
}

export function crearStore() {
  const sagaMiddleware = createSagaMiddleware()
```

```
  const store = configureStore({
    reducer: { patrones: patronesSlice.reducer },
    middleware: (getDefault) => getDefault({ thunk: false
      }).concat(sagaMiddleware),
  })
  sagaMiddleware.run(rootSaga)
  return store
}

export type RootState = ReturnType<ReturnType<typeof crearStore>
  ['getState']>
```

race: timeout

```
const { respuesta, timeout } = yield race({
  respuesta: call(api.lenta),
  timeout: delay(2000, true),
})
```

`race` corre varios efectos y se queda con **el primero que termina**. Los demás se **cancelan** solos. Tocá el botón varias veces: la API tarda entre 0,5 y 3 segundos, así que a veces gana la respuesta y a veces el timeout.

El mismo patrón sirve para "cancelar si el usuario toca Cancelar": `race({ resultado: call(...), cancelar: take('CANCELADO') })`.

fork, take y cancel: polling

```
function* controlarPolling() {
  while (true) {
    yield take('pollingIniciado') // espera
    const tarea = yield fork(polling) // arranca en paralelo
    yield take('pollingDetenido') // espera
    yield cancel(tarea) // corta el loop, esté donde esté
  }
}
```

Una saga con `while (true)` que hace `take` es un **flujo**: se lee de arriba abajo como una historia ("esperá que empiece, arrancá,

esperá que pare, cortá"). Con thunks, esto requeriría guardar el id de un `setInterval` en algún lado y acordarse de limpiarlo.

Manejo de errores

```
function* cargar() {
  try {
    const datos = yield call(api.listar)
    yield put(cargadas(datos))
  } catch (error) {
    yield put(falloLaCarga(String(error)))
  }
}
```

Un error en un `call` se tira **adentro de la saga**, en la línea del `yield`, así que se maneja con un `try/catch` común. Ojo: un error sin atrapar en una saga hija puede terminar con la saga raíz y dejar de escuchar acciones.

Testear una saga

Como los efectos son objetos, se testea recorriendo el generador:

```
const gen = cargarSaga()
expect(gen.next().value).toEqual(put(cargando()))
expect(gen.next().value).toEqual(call(api.listar))
const datos = [{ id: 1, empresa: 'Acme', etapa: 'postulado' }]
expect(gen.next(datos).value).toEqual(put(cargadas(['Acme']))) // le
  "inyectás" la respuesta
expect(gen.next().done).toBe(true)
```

No hay que mockear la API: nunca se llama. Le pasás a `next()` lo que habría devuelto.

¿Saga hoy?

⚠️ Para proyectos nuevos, casi nunca

El equipo de Redux recomienda **no** usar Saga para lo común: para traer datos, **RTK Query**; para "cuando pase X, hacé Y", el **listener middleware** de Redux Toolkit, que cubre la mayoría de los casos (incluida la cancelación) con async/await normal. Saga sigue siendo muy buena para flujos largos y complejos, pero suma una curva de aprendizaje (generadores, efectos) y tipos más débiles en TypeScript (cada `yield` hay que anotarlo a mano).

Lo importante para vos: **saber leerla y mantenerla**, porque está en muchísimo código que todavía corre en producción.

🎤 En una entrevista

"¿Por qué alguien elegiría Saga en vez de thunks?" → "Para flujos async complejos: cancelación (takeLatest, cancel), carreras con timeout (race), procesos de fondo como polling (fork) y flujos que esperan acciones del usuario (take). Además los efectos son datos, así que se testean sin mocks. Para casos simples, hoy usaría RTK Query o el listener middleware."

Parte 4 · Zustand

Un store global en un hook, sin Provider ni boilerplate.

Capítulo 10

Zustand

create, set y get, selectores, useShallow, persist y cómo organizar un store grande.

Zustand ("estado" en alemán) es un store global en un hook. Sin Provider, sin acciones con `type`, sin reducers: el estado y las funciones que lo cambian viven juntos, y cada componente elige qué parte leer.

```
const useContador = create<{ cuenta: number; sumar: () => void }>()
  ((set) => ({
  cuenta: 0,
  sumar: () => set((s) => ({ cuenta: s.cuenta + 1 })),
}))

function Boton() {
  const sumar = useContador((s) => s.sumar)
  return <button onClick={sumar}>+1</button>
}
```

Eso es todo. Es la librería de estado global que más creció en los últimos años, sobre todo en proyectos nuevos.

DEMO **Un store de Zustand con acciones async, selectores y persist**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/estado/Zustand.tsx

```tsx
import { useState } from 'react'
import { useShallow } from 'zustand/react/shallow'
import { useRenderFlash } from '../shared/useRenderFlash'
import { totalActual, usePostulaciones } from './zustand/store'

export default function Zustand() {
  return (
    <div className="stack">
      <Controles />
      <div className="grid-2">
        <Lista />
        <Total />
      </div>
      <p className="muted">Recargá la página: las postulaciones siguen
        ahí (middleware persist).</p>
    </div>
  )
}

function Controles() {
  // Un selector por valor: el componente solo se re-renderiza si
    cambia lo que elegiste.
  const status = usePostulaciones((s) => s.status)
  const filtro = usePostulaciones((s) => s.filtro)
  const { cargar, setFiltro, reiniciar } = usePostulaciones(
    // useShallow: elegir varias cosas en un objeto sin re-renderizar
      si son las mismas.
    useShallow((s) => ({ cargar: s.cargar, setFiltro: s.setFiltro,
      reiniciar: s.reiniciar })),
  )
  const [leido, setLeido] = useState<number | null>(null)
  return (
    <div className="row">
      <button onClick={() => cargar()}>Cargar</button>
      <button onClick={() => cargar(true)}>Cargar (con error)</button>
      <button onClick={reiniciar}>Vaciar</button>
      <button onClick={() => setLeido(totalActual())}>getState(){leido
        !== null && ` → ${leido}`}</button>
      <span className="pill">status: {status}</span>
      <input value={filtro} onChange={(e) =>
        setFiltro(e.target.value)} placeholder="Filtrar" />
    </div>
  )
}

function Lista() {
  const items = usePostulaciones((s) => s.items)
  const filtro = usePostulaciones((s) => s.filtro)
```

```
  const avanzar = usePostulaciones((s) => s.avanzar)
  const ref = useRenderFlash<HTMLDivElement>()
  const filtradas = items.filter((p) =>
    p.empresa.toLowerCase().includes(filtro.toLowerCase()))
  return (
    <div ref={ref} className="card">
      <ul>
        {filtradas.map((p) => (
          <li key={p.id}>
            {p.empresa} · {p.etapa} <button onClick={() =>
              avanzar(p.id)}>avanzar</button>
          </li>
        ))}
      </ul>
      {items.length === 0 && <p className="muted">Vacío: tocá Cargar.
        </p>}
    </div>
  )
}

function Total() {
  const total = usePostulaciones((s) => s.items.length)
  const ref = useRenderFlash<HTMLDivElement>()
  return (
    <div ref={ref} className="card">
      <strong>Total</strong>
      <p style={{ fontSize: '2rem', margin: 0 }}>{total}</p>
    </div>
  )
}
```

src/demos/estado/zustand/store.ts

```
import { create } from 'zustand'
import { createJSONStorage, persist } from 'zustand/middleware'
import { api, siguienteEtapa, type Postulacion } from '../api'

type PostulacionesStore = {
  // Estado
  items: Postulacion[]
  filtro: string
  status: 'idle' | 'loading' | 'failed'
  // Acciones: viven en el mismo store, al lado de los datos
  cargar: (falla?: boolean) => Promise<void>
  avanzar: (id: number) => void
  setFiltro: (filtro: string) => void
  reiniciar: () => void
}
```

```
// create devuelve un hook. No hay Provider, ni acciones con type, ni
  reducer.
export const usePostulaciones = create<PostulacionesStore>()(
  // persist guarda el estado en localStorage y lo recupera al
    recargar.
  persist(
    (set, get) => ({
      items: [],
      filtro: '',
      status: 'idle',

      async cargar(falla = false) {
        if (get().status === 'loading') return // get() lee el estado
          actual
        set({ status: 'loading' }) // set hace un merge superficial:
          no hace falta ...state
        try {
          set({ items: await api.listar({ falla }), status: 'idle' })
        } catch {
          set({ status: 'failed' })
        }
      },

      avanzar: (id) =>
        set((state) => ({
          items: state.items.map((p) => (p.id === id ? { ...p, etapa:
            siguienteEtapa[p.etapa] } : p)),
        })),

      setFiltro: (filtro) => set({ filtro }),

      reiniciar: () => set({ items: [], filtro: '', status: 'idle' }),
    }),
    {
      name: 'guia-estado-zustand', // la clave en localStorage
      storage: createJSONStorage(() => localStorage),
      partialize: (state) => ({ items: state.items }), // solo
        persistimos las postulaciones
    },
  ),
)

// Fuera de React también se puede leer, escribir y suscribirse:
export const totalActual = () =>
  usePostulaciones.getState().items.length
```

Cargá la lista, filtrá y avanzá postulaciones. Mirá qué parpadea: al escribir en el filtro, `Total` no se re-renderiza porque solo lee

`items.length`. Y **recargá la página**: las postulaciones siguen ahí gracias al middleware `persist`.

Lo esencial

- `create<T>()(fn)`: los paréntesis extra son para que TypeScript infiera bien con middlewares. `fn` recibe `set` y `get` y devuelve el estado inicial con las acciones.
- `set(parcial)` hace un *merge* superficial: `set({ filtro })` no borra el resto. Con una función, `set((s) => ...)` recibe el estado actual.
- `get()` lee el estado actual adentro de una acción (útil en las async).
- **Acciones async**: son funciones `async` comunes. No hace falta middleware.
- **Fuera de React**: `useStore.getState()`, `useStore.setState()` y `useStore.subscribe()`. Sirve para usarlo desde un interceptor de axios, un websocket o un test.

Selectores y re-renders

El hook recibe un selector, y el componente se re-renderiza solo si el resultado cambia (con `Object.is`), igual que `useSelector`:

```
const items = usePostulaciones((s) => s.items) // ✅ solo cuando
  cambian los items
const todo = usePostulaciones() // ⚠️ con cualquier cambio del store
```

> **⚠️ Elegir varias cosas: useShallow**
>
> Un selector que arma un objeto o array nuevo (`(s) => ({ a: s.a, b: s.b })`) devuelve algo distinto en cada llamada. En Zustand 5 eso provoca un **loop infinito de renders** ("Maximum update depth exceeded"). Envolvelo en `useShallow`, que compara las propiedades una por una, o usá un selector por valor.

Middlewares

Middleware	Para qué
`persist`	Guardar en `localStorage` (o donde quieras) y rehidratar al cargar. `partialize` elige qué guardar.
`devtools`	Conectar con la extensión Redux DevTools.
`immer`	Escribir `set((s) => { s.items.push(x) })` mutando un borrador, como en RTK.
`subscribeWithSelector`	Suscribirse solo a una parte del estado fuera de React.

Se componen anidándolos: `create<T>()(devtools(persist((set) => ({ ... }), { name: 'x' })))`.

Organizar un store grande

Un solo `create` por dominio (`useCarrito`, `useFiltros`, `useSesion`) suele ser suficiente. Si un store crece, se puede partir en **slices**: funciones que reciben `set` / `get` y devuelven una parte del estado, combinadas en un solo `create`.

> ℹ️ **Zustand y Next.js**
>
> Un store creado con `create` a nivel de módulo es **global**: en el servidor de Next.js lo compartirían todos los requests. Para el App Router, la documentación recomienda crear el store por request con `createStore` (de `zustand/vanilla`) y pasarlo por un context. En apps solo de cliente, el `create` global está perfecto.

🎤 En una entrevista

"¿Por qué Zustand y no Redux?" → "Para estado global del cliente de tamaño chico o mediano, Zustand es mucho más simple: un hook con estado y acciones, sin Provider ni boilerplate, con selectores para evitar re-renders y middlewares como persist o devtools. Redux Toolkit lo elegiría en apps grandes con mucha lógica compartida, equipos grandes que se benefician de la estructura, o si necesito RTK Query y las DevTools con time travel."

Parte 5 · Comparar y decidir

La misma feature con las cuatro herramientas, una tabla para decidir y la chuleta.

Capítulo 11

Las cuatro, lado a lado

La misma feature en Redux, Redux Toolkit, Saga y Zustand, cómo elegir y cómo migrar.

La misma feature, cuatro veces: cargar una lista de la API (con estado de carga), agregar un elemento y quitarlo. Así se ve la diferencia real de cada herramienta, no en teoría.

DEMO **Redux clásico, Redux Toolkit, RTK + Saga y Zustand lado a lado**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/estado/Comparacion.tsx`

```
import ConRedux from './comparacion/ConRedux'
import redux from './comparacion/ConRedux.tsx?raw'
import ConSaga from './comparacion/ConSaga'
import saga from './comparacion/ConSaga.tsx?raw'
import ConToolkit from './comparacion/ConToolkit'
import toolkit from './comparacion/ConToolkit.tsx?raw'
import ConZustand from './comparacion/ConZustand'
import zustand from './comparacion/ConZustand.tsx?raw'

// Cuenta las líneas con código (sin vacías ni comentarios) del
  archivo fuente real.
const lineas = (fuente: string) => fuente.split('\n').filter((l) =>
  l.trim() && !l.trim().startsWith('//')).length
```

```
const versiones = [
  { nombre: 'Redux clásico', Componente: ConRedux, fuente: redux },
  { nombre: 'Redux Toolkit', Componente: ConToolkit, fuente: toolkit
    },
  { nombre: 'RTK + Saga', Componente: ConSaga, fuente: saga },
  { nombre: 'Zustand', Componente: ConZustand, fuente: zustand },
]

export default function Comparacion() {
  return (
    <div className="stack">
      <p className="muted">
        La misma feature cuatro veces: cargar favoritas de la API,
          agregar una y quitar (tocá un nombre).
      </p>
      <div className="grid-2">
        {versiones.map(({ nombre, Componente, fuente }) => (
          <section key={nombre} className="card">
            <div className="row">
              <strong style={{ flex: 1 }}>{nombre}</strong>
              <span className="pill">{lineas(fuente)} líneas</span>
            </div>
            <Componente />
          </section>
        ))}
      </div>
    </div>
  )
}
```

src/demos/estado/comparacion/ConRedux.tsx

```
// Redux "clásico": constantes, action creators, reducer con switch,
  thunk a mano.
import { useState } from 'react'
import { Provider, useDispatch, useSelector } from 'react-redux'
import { applyMiddleware, legacy_createStore as createStore } from
  'redux'
import { thunk, type ThunkAction, type ThunkDispatch } from 'redux-
  thunk'
import { api } from '../api'

const CARGADAS = 'favoritas/cargadas'
const AGREGADA = 'favoritas/agregada'
const QUITADA = 'favoritas/quitada'

type State = { lista: string[]; cargando: boolean }
type Action =
  | { type: typeof CARGADAS; payload: string[] }
  | { type: typeof AGREGADA; payload: string }
```

```
  | { type: typeof QUITADA; payload: string }
  | { type: 'favoritas/cargando' }

const agregar = (empresa: string): Action => ({ type: AGREGADA,
  payload: empresa })
const quitar = (empresa: string): Action => ({ type: QUITADA, payload:
  empresa })
const cargar = (): ThunkAction<Promise<void>, State, unknown, Action>
  => async (dispatch) => {
  dispatch({ type: 'favoritas/cargando' })
  dispatch({ type: CARGADAS, payload: (await api.listar()).map((p) =>
    p.empresa) })
}

function reducer(state: State = { lista: [], cargando: false },
  action: Action): State {
  switch (action.type) {
    case 'favoritas/cargando':
      return { ...state, cargando: true }
    case CARGADAS:
      return { lista: action.payload, cargando: false }
    case AGREGADA:
      return { ...state, lista: [...state.lista, action.payload] }
    case QUITADA:
      return { ...state, lista: state.lista.filter((e) => e !==
        action.payload) }
    default:
      return state
  }
}

export default function ConRedux() {
  const [store] = useState(() => createStore(reducer,
    applyMiddleware(thunk)))
  return (
    <Provider store={store}>
      <Favoritas />
    </Provider>
  )
}

function Favoritas() {
  const { lista, cargando } = useSelector((s: State) => s)
  const dispatch = useDispatch<ThunkDispatch<State, unknown, Action>>
    ()
  return (
    <>
      <button onClick={() => dispatch(cargar())}>{cargando ?
        'Cargando…' : 'Cargar'}</button>
      <button onClick={() => dispatch(agregar('Mate Cloud'))}>+ Mate
        Cloud</button>
      <ul>
```

```
        {lista.map((e) => (
          <li key={e} onClick={() => dispatch(quitar(e))}>
            {e}
          </li>
        ))}
      </ul>
    </>
  )
}
```

src/demos/estado/comparacion/ConToolkit.tsx

```
// Redux Toolkit: createSlice + createAsyncThunk. Mismo Redux, mucho
  menos código.
import { configureStore, createAsyncThunk, createSlice, type
  PayloadAction } from '@reduxjs/toolkit'
import { useState } from 'react'
import { Provider, useDispatch, useSelector } from 'react-redux'
import { api } from '../api'

const cargar = createAsyncThunk('favoritas/cargar', async () => (await
  api.listar()).map((p) => p.empresa))

const favoritas = createSlice({
  name: 'favoritas',
  initialState: { lista: [] as string[], cargando: false },
  reducers: {
    agregar: (state, action: PayloadAction<string>) => void
      state.lista.push(action.payload),
    quitar: (state, action: PayloadAction<string>) => {
      state.lista = state.lista.filter((e) => e !== action.payload)
    },
  },
  extraReducers: (builder) => {
    builder.addCase(cargar.pending, (state) => void (state.cargando =
      true))
    builder.addCase(cargar.fulfilled, (_state, action) => ({ lista:
      action.payload, cargando: false }))
  },
})
const { agregar, quitar } = favoritas.actions

const crearStore = () => configureStore({ reducer: favoritas.reducer
  })
type State = ReturnType<typeof favoritas.reducer>
type AppDispatch = ReturnType<typeof crearStore>['dispatch']

export default function ConToolkit() {
  const [store] = useState(crearStore)
  return (
```

```
    <Provider store={store}>
      <Favoritas />
    </Provider>
  )
}

function Favoritas() {
  const { lista, cargando } = useSelector((s: State) => s)
  const dispatch = useDispatch<AppDispatch>()
  return (
    <>
      <button onClick={() => dispatch(cargar())}>{cargando ?
        'Cargando…' : 'Cargar'}</button>
      <button onClick={() => dispatch(agregar('Mate Cloud'))}>+ Mate
        Cloud</button>
      <ul>
        {lista.map((e) => (
          <li key={e} onClick={() => dispatch(quitar(e))}>
            {e}
          </li>
        ))}
      </ul>
    </>
  )
}
```

src/demos/estado/comparacion/ConSaga.tsx

```
// Redux Toolkit + Redux Saga: el slice guarda datos, la saga se
  encarga de lo async.
import { configureStore, createAction, createSlice, type PayloadAction
  } from '@reduxjs/toolkit'
import { useState } from 'react'
import { Provider, useDispatch, useSelector } from 'react-redux'
import createSagaMiddleware from 'redux-saga'
import { call, put, takeLatest } from 'redux-saga/effects'
import { api, type Postulacion } from '../api'

const favoritas = createSlice({
  name: 'favoritas',
  initialState: { lista: [] as string[], cargando: false },
  reducers: {
    cargando: (state) => void (state.cargando = true),
    cargadas: (_state, action: PayloadAction<string[]>) => ({ lista:
      action.payload, cargando: false }),
    agregar: (state, action: PayloadAction<string>) => void
      state.lista.push(action.payload),
    quitar: (state, action: PayloadAction<string>) => {
      state.lista = state.lista.filter((e) => e !== action.payload)
    },
```

```
  },
})
const { cargando, cargadas, agregar, quitar } = favoritas.actions
const cargar = createAction('favoritas/cargar') // solo la escucha la
  saga

function* cargarSaga() {
  yield put(cargando())
  const postulaciones: Postulacion[] = yield call(api.listar)
  yield put(cargadas(postulaciones.map((p) => p.empresa)))
}

function crearStore() {
  const saga = createSagaMiddleware()
  const store = configureStore({ reducer: favoritas.reducer,
    middleware: (d) => d().concat(saga) })
  saga.run(function* () {
    yield takeLatest(cargar.type, cargarSaga)
  })
  return store
}
type State = ReturnType<typeof favoritas.reducer>

export default function ConSaga() {
  const [store] = useState(crearStore)
  return (
    <Provider store={store}>
      <Favoritas />
    </Provider>
  )
}

function Favoritas() {
  const { lista, cargando } = useSelector((s: State) => s)
  const dispatch = useDispatch()
  return (
    <>
      <button onClick={() => dispatch(cargar())}>{cargando ?
        'Cargando…' : 'Cargar'}</button>
      <button onClick={() => dispatch(agregar('Mate Cloud'))}>+ Mate
        Cloud</button>
      <ul>
        {lista.map((e) => (
          <li key={e} onClick={() => dispatch(quitar(e))}>
            {e}
          </li>
        ))}
      </ul>
    </>
  )
}
```

src/demos/estado/comparacion/ConZustand.tsx

```tsx
// Zustand: un hook con el estado y las acciones juntos. Sin Provider
  ni acciones con type.
import { create } from 'zustand'
import { api } from '../api'

type Favoritas = {
  lista: string[]
  cargando: boolean
  cargar: () => Promise<void>
  agregar: (empresa: string) => void
  quitar: (empresa: string) => void
}

const useFavoritas = create<Favoritas>()((set) => ({
  lista: [],
  cargando: false,
  cargar: async () => {
    set({ cargando: true })
    set({ lista: (await api.listar()).map((p) => p.empresa), cargando:
      false })
  },
  agregar: (empresa) => set((s) => ({ lista: [...s.lista, empresa]
    })),
  quitar: (empresa) => set((s) => ({ lista: s.lista.filter((e) => e
    !== empresa) })),
}))

export default function ConZustand() {
  const { lista, cargando, cargar, agregar, quitar } = useFavoritas()
  return (
    <>
      <button onClick={cargar}>{cargando ? 'Cargando…' : 'Cargar'}
        </button>
      <button onClick={() => agregar('Mate Cloud')}>+ Mate
        Cloud</button>
      <ul>
        {lista.map((e) => (
          <li key={e} onClick={() => quitar(e)}>
            {e}
          </li>
        ))}
      </ul>
    </>
  )
}
```

El número de líneas sale del archivo fuente real (sin vacías ni comentarios). Mirá las pestañas: la lógica es exactamente la misma, lo que cambia es cuánto código hace falta para expresarla y dónde vive cada parte.

Con una sola feature la diferencia parece chica. Pero en Redux clásico cada acción nueva suma una constante, un action creator y un `case`, y cada operación async tres acciones más; en Redux Toolkit es una función en el slice, y en Zustand, una función en el store. En una app con decenas de features, eso es lo que separa un archivo de 50 líneas de uno de 300.

La tabla

	Redux clásico	Redux Toolkit	RTK + Saga	Zustand
Boilerplate	Mucho	Poco	Medio	Mínimo
Provider	Sí	Sí	Sí	No
Lógica async	Thunks a mano	`createAsyncThunk`, listener	Sagas (generadores)	Funciones async en el store
Datos del servidor	A mano	RTK Query	A mano o RTK Query	TanStack Query aparte
Re-renders	Selectores + `createSelector`	Igual	Igual	Selectores + `useShallow`
DevTools / time travel	Sí	Sí, configurado	Sí	Con el middleware `devtools`
TypeScript	Verboso	Muy bueno (inferencia)	Débil en los `yield`	Muy bueno
Curva de aprendizaje	Media	Media	Alta	Baja
Tamaño (min+gzip aprox.)	~5 kB con react-redux	~15 kB (sin RTK Query)	+~5 kB de Saga	~1 kB

	Redux clásico	Redux Toolkit	RTK + Saga	Zustand
Cuándo	Mantener código legacy	Apps medianas y grandes, equipos grandes	Flujos async complejos en código existente	Apps chicas y medianas, estado del cliente simple

Cómo decidir

1. **¿Son datos del servidor?** Usá una librería de caché: **TanStack Query** o **RTK Query** (si ya hay Redux). Esto solo ya saca la mayor parte de lo que antes iba al store.
2. **¿Queda estado global del cliente?** Si es poco y simple (filtros, preferencias, un carrito), **Zustand** o incluso **Context**. Si es mucho, con lógica compleja, muchas personas tocando el código y necesidad de trazabilidad, **Redux Toolkit**.
3. **¿Hay flujos async muy complejos** (cancelación, polling, pasos que esperan al usuario)? Primero probá el **listener middleware** de RTK. Si ya hay Saga en el proyecto, seguí con Saga.
4. **¿Proyecto legacy con Redux clásico?** No reescribas todo: **migrá de a un slice** a Redux Toolkit. `configureStore` acepta los reducers viejos tal como están, así que los dos estilos conviven.

Migrar Redux clásico a Redux Toolkit

Un camino que funciona en proyectos reales, sin frenar features:

1. Reemplazá `createStore` + `combineReducers` + `applyMiddleware` por `configureStore`, pasándole los reducers viejos. Nada más cambia.
2. Cada vez que toques una feature, convertí su reducer en un `createSlice` (las acciones nuevas tienen el mismo `type` si usás el mismo nombre).
3. Pasá los thunks a `createAsyncThunk`, y los que solo traen datos, a endpoints de RTK Query.
4. Cambiá `connect` por hooks en los componentes que vayas tocando.

5. Si hay sagas, dejalas: siguen funcionando con `configureStore`. Las nuevas features pueden usar el listener middleware.

> **🎤 En una entrevista**
>
> "Si arrancás un proyecto hoy, ¿qué usás para el estado?" → "Separo el estado del servidor del estado del cliente. Para el servidor, TanStack Query o RTK Query. Para el cliente, empiezo con useState, context o Zustand si es poco; si la app es grande, con mucha lógica compartida y un equipo grande, Redux Toolkit. Saga solo si el proyecto ya la usa o hay flujos async muy complejos."

Capítulo 12

Chuleta y preguntas de entrevista

Las APIs de las cuatro librerías en snippets cortos y respuestas para entrevistas.

Todo en una página, para repasar antes de una entrevista o para tener a mano en un proyecto.

Redux clásico

```
const store = createStore(reducer, applyMiddleware(thunk))
store.dispatch({ type: 'postulaciones/agregada', payload: p })
store.getState()
store.subscribe(() => {})

function reducer(state = inicial, action) {
  switch (action.type) {
    case 'postulaciones/agregada':
      return { ...state, items: [...state.items, action.payload] }
    default:
      return state
  }
}

// Thunk
const cargar = () => async (dispatch, getState) => {
  dispatch({ type: 'cargando' })
  dispatch({ type: 'cargadas', payload: await api.listar() })
}

// connect (legacy)
export default connect((state) => ({ total: state.items.length }), {
  agregar })(Componente)
```

React Redux

```
<Provider store={store}><App /></Provider>

const items = useAppSelector((s) => s.postulaciones.items)
const dispatch = useAppDispatch()

const selectFiltradas = createSelector([selectItems, selectFiltro],
  (items, f) => items.filter(...))

export const useAppSelector = useSelector.withTypes<RootState>()
export const useAppDispatch = useDispatch.withTypes<AppDispatch>()
```

Redux Toolkit

```
const slice = createSlice({
  name: 'postulaciones',
  initialState,
  reducers: {
    agregada(state, action: PayloadAction<Postulacion>) {
      state.items.push(action.payload) // Immer
    },
  },
  extraReducers: (builder) => {
    builder.addCase(cargar.fulfilled, (state, action) => {
      state.items = action.payload
    })
  },
})

const cargar = createAsyncThunk('postulaciones/cargar', () =>
  api.listar())
await dispatch(cargar()).unwrap()

const store = configureStore({ reducer: { postulaciones: slice.reducer
  } })
type RootState = ReturnType<typeof store.getState>
type AppDispatch = typeof store.dispatch
```

RTK Query

```
const api = createApi({
```

```
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  tagTypes: ['Oferta'],
  endpoints: (b) => ({
    listar: b.query<Oferta[], void>({ query: () => 'ofertas',
      providesTags: ['Oferta'] }),
    crear: b.mutation<Oferta, Nueva>({ query: (body) => ({ url:
      'ofertas', method: 'POST', body }), invalidatesTags: ['Oferta']
      }),
  }),
})
const { data, isLoading, isFetching, error, refetch } =
  api.useListarQuery()
const [crear, { isLoading: creando }] = api.useCrearMutation()
```

Redux Saga

```
function* buscar(action) {
  yield delay(300) // debounce con takeLatest
  const datos = yield call(api.buscar, action.payload)
  yield put(resultados(datos))
}
function* root() {
  yield takeLatest('busqueda/cambiada', buscar)
}
// race, fork, cancel, take, select
const { ok, timeout } = yield race({ ok: call(api.lenta), timeout:
  delay(2000) })
```

Zustand

```
const useStore = create<State>()(
  persist(
    (set, get) => ({
      items: [],
      agregar: (p) => set((s) => ({ items: [...s.items, p] })),
      cargar: async () => set({ items: await api.listar() }),
    }),
    { name: 'clave' },
  ),
)
const items = useStore((s) => s.items)
const { a, b } = useStore(useShallow((s) => ({ a: s.a, b: s.b })))
useStore.getState().agregar(p) // fuera de React
```

Preguntas de entrevista

¿Qué problema resuelve Redux? Centraliza el estado global en un store predecible: solo cambia despachando acciones que procesan reducers puros. Facilita compartir estado entre partes lejanas del árbol, depurar (DevTools, time travel) y testear.

¿Qué es un reducer y por qué tiene que ser puro? Una función `(estado, acción) => estadoNuevo`. Tiene que ser pura (sin mutar ni efectos) para que el resultado dependa solo de sus entradas: así Redux detecta cambios por referencia y se puede reproducir cualquier estado.

¿Qué es un middleware? Una función entre el dispatch y el reducer que ve cada acción. Ahí va lo async (thunk, saga), el logging o la analítica.

¿Thunk o Saga? Thunk: async/await simple, ideal para la mayoría. Saga: flujos complejos con cancelación, carreras y procesos de fondo, testeables sin mocks, con más curva de aprendizaje.

¿Qué aporta Redux Toolkit? Menos boilerplate (`createSlice`, `createAsyncThunk`), Immer para "mutar" de forma segura, `configureStore` con buenos defaults y RTK Query para la caché de datos del servidor.

¿Cómo evitás re-renders con useSelector? Seleccionando lo mínimo y memorizando con `createSelector` los selectores que devuelven arrays u objetos nuevos.

¿Redux, Context o Zustand? Context es para inyectar valores que cambian poco (tema, sesión): todos los consumidores se re-renderizan con cada cambio. Zustand para estado global simple con selectores. Redux Toolkit para apps grandes con mucha lógica y necesidad de estructura.

¿Dónde guardás los datos que vienen de la API? En una librería de caché (RTK Query o TanStack Query), no a mano en el store.

¿Cómo migrarías un Redux legacy? De a poco: `configureStore` con los reducers viejos, después cada feature a `createSlice`, thunks a

`createAsyncThunk` o RTK Query, y `connect` a hooks cuando se toque cada componente.